

FoldAudit: Sound Polynomial Folding for Dynamic Multi-File Data Integrity Auditing

Abstract—

Index Terms—Polynomial Commitment, Vector Commitment, Data Management

I. INTRODUCTION

The exponential growth of global data generation has driven a paradigm shift toward decentralized and cloud-based storage architectures. By outsourcing data to remote storage providers, individuals and organizations significantly reduce local hardware maintenance costs while benefiting from elastic scalability, geographic redundancy, and high availability. However, this outsourcing model inherently relinquishes physical control over the stored assets, exposing them to multifaceted risks such as hardware degradation, silent data corruption, and economically motivated data deletion or compression by untrusted storage providers. To mitigate these threats and restore trust in distributed storage ecosystems, Provable Data Possession (PDP) and Proof of Retrievability (PoR) schemes have emerged as foundational cryptographic primitives. These protocols enable data owners or delegated auditors to efficiently verify the integrity of outsourced data without retrieving the entire dataset, thereby ensuring continuous data availability, regulatory compliance, and accountability in modern storage networks.

Recent advancements in cryptographic primitives have significantly optimized the efficiency and scalability of data integrity auditing. Among these, Polynomial Commitment Schemes (PCS), particularly the Kate-Zaverucha-Goldberg (KZG) construction, have been widely adopted in state-of-the-art PDP designs. By encoding data blocks as coefficients of low-degree polynomials over a finite field, PCS enables compact, constant-size commitments and supports homomorphic aggregation of multiple block tags. This algebraic structure allows cloud servers to generate succinct proofs of possession through polynomial evaluation and quotient computation, while verifiers can validate correctness using a minimal number of bilinear pairings. Consequently, polynomial commitments have become the cornerstone of lightweight, publicly verifiable auditing frameworks, drastically reducing communication bandwidth and on-chain storage overhead compared to traditional hash-based or signature-based approaches. Their compatibility with structured reference strings (SRS) and efficient synthetic division algorithms further positions them as ideal candidates for large-scale storage verification.

Despite these cryptographic improvements, practical storage deployments frequently involve massive-scale data repositories comprising thousands of files distributed across multiple storage nodes, providers, or geographic regions. Conventional

PDP protocols process each file independently, requiring separate proof generation, transmission, and verification rounds for every audited dataset. This linear scaling introduces severe computational and economic bottlenecks, particularly in blockchain-assisted auditing systems where smart contract execution costs (gas) and on-chain proof storage scale directly with the number of files. As audit frequencies increase and batch sizes grow, the cumulative overhead of independent verifications rapidly degrades system throughput, inflates operational expenses, and strains verifier resources. Therefore, developing a secure and efficient multi-file batch verification mechanism that compresses multiple integrity checks into a single, constant-size cryptographic operation has become a critical research imperative for scalable decentralized storage.

Given the substantial size of individual files in real-world applications, auditing every data block is computationally prohibitive and economically unviable. To address this challenge, modern PDP schemes universally employ probabilistic sampling strategies. During an audit round, the verifier randomly selects a subset of data blocks and assigns independent random coefficients to construct a linear combination of challenged sectors. This approach reduces the proof size and verification complexity from linear $O(n)$ to constant or logarithmic $O(c)$ with respect to the total number of blocks, where $c \ll n$ denotes the challenge size. By carefully calibrating the number of sampled blocks, auditors can achieve arbitrarily high detection probabilities for data corruption while maintaining minimal communication and computation overhead. The randomness of the challenge set ensures that malicious providers cannot precompute proofs or selectively store only frequently accessed blocks without facing a high risk of detection.

However, the introduction of probabilistic sampling and homomorphic aggregation introduces subtle yet critical privacy vulnerabilities. In many existing batch auditing designs, challenge generation mechanisms, evaluation points, and masking techniques are either oversimplified or structurally coupled across files. If challenge points are shared, folding weights are predictable, or masking values are reused, an honest-but-curious verifier or public ledger observer can exploit the linear algebraic structure of aggregated evaluations. Through cross-round correlation analysis or linear system inversion on publicly disclosed challenge-response transcripts, adversaries may reconstruct underlying polynomial evaluations $f(r_i)$ or even infer sensitive sector values. Such privacy leakage fundamentally contradicts the confidentiality requirements of modern cloud and decentralized storage, highlighting the necessity of integrating fresh one-time masking directly into the batch verification algebraic structure.

To address the aforementioned challenges, we propose **FoldAudit**, a novel batch-verifiable PDP scheme that si-

multaneously achieves efficient multi-file verification, tight cryptographic soundness, and practical evaluation masking. FoldAudit introduces a layered algebraic folding architecture: (i) independent per-file challenge points $\{r_i\}$ to structurally isolate file-specific quotient polynomials and prevent cross-file pole cancellation; (ii) a Fiat-Shamir derived random folding scalar ρ applied after the proof statement is fixed, ensuring that batch aggregation remains algebraically unpredictable and bias-resistant; and (iii) additive one-time masking $\tilde{y}_i = y_i + \mu_i$ with a public mask commitment $R_i = g^{\mu_i}$ and a Schnorr proof of knowledge for extractability. The folded KZG relation yields a soundness error bounded by $\epsilon \leq \frac{m-1}{p} + \text{negl}(\lambda)$ for challenged corrupted data, guaranteeing that adversarial cross-file cancellation is detected with overwhelming probability. Theoretical analysis and experimental evaluations demonstrate that FoldAudit keeps verification dominated by challenged-tag aggregation, lightweight Schnorr mask checks, and a single folded pairing check, supports stateless dynamic file updates without global state recomputation, and provides privacy when challenged evaluations have sufficient min-entropy against enumeration by honest-but-curious auditors.

II. PRELIMINARIES

A. Notations

For clarity and reference, we summarize the primary mathematical symbols, cryptographic parameters, and protocol-specific variables used throughout the paper in Table I.

B. Basics of Polynomials

Let $\mathbb{F}_p$ be a finite field of prime order p , where p is super-polynomial in the security parameter λ . For an integer $d > 0$, let $\mathbb{F}_p^{<d}[X]$ denote the set of univariate polynomials over $\mathbb{F}_p$ of degree strictly smaller than d . We write $[m] = \{1, \dots, m\}$ and use uppercase letters for indeterminates (e.g., X) and lowercase letters for field elements (e.g., $x, r, z \in \mathbb{F}_p$). All polynomial identities in this paper are over $\mathbb{F}_p[X]$, and all exponents of group elements are interpreted modulo p .

a) *Quotient Polynomials.*: For $f(X) \in \mathbb{F}_p[X]$ and $r, y \in \mathbb{F}_p$, the polynomial remainder theorem implies

$$f(X) - y = (X - r)w(X) + (f(r) - y)$$

for a unique quotient polynomial $w(X) \in \mathbb{F}_p[X]$. Hence $(X - r)$ divides $f(X) - y$ if and only if $y = f(r)$, in which case

$$w(X) = \frac{f(X) - y}{X - r}$$

is well-defined and satisfies $\deg(w) \leq \deg(f) - 1$. FoldAudit uses this divisibility relation as the algebraic statement certified by each per-file KZG witness.

Lemma 1 (Schwartz–Zippel). *Let $P(X) \in \mathbb{F}_p[X]$ be a non-zero polynomial of degree at most d . For uniformly sampled $\rho \leftarrow \mathbb{F}_p$,*

$$\Pr[P(\rho) = 0] \leq \frac{d}{p}.$$

Table I: Summary of Notation and System Parameters

Symbol	Description
λ	Security parameter.
p	Prime order of the finite field $\mathbb{F}_p$ and bilinear groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$.
$[m]$	Index set $\{1, 2, \dots, m\}$ for the audited file batch.
$\mathbb{F}_p^{<d}[X]$	Vector space of univariate polynomials over $\mathbb{F}_p$ with degree strictly less than d .
g, h	Public generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively.
$e(\cdot, \cdot)$	Efficiently computable, non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.
τ	Secret trapdoor used for Structured Reference String (SRS) generation (held by DO).
SRS	Public parameters $(g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$ supporting polynomial degrees $< s$.
H_1	Collision-resistant hash function for Merkle tree construction.
H_2	Fiat-Shamir random oracle for transcript binding and deterministic challenge derivation.
m, n, s, c	Number of files, chunks per file, sectors per chunk, and challenged chunks per file.
i, j, k	Indices for files ($i \in [m]$), chunks ($j \in [n]$), and sectors ($0 \leq k < s$).
$m_{i,j,k}$	k -th sector value of chunk j in file i .
$f_{i,j}(X)$	Polynomial encoding of chunk j in file i : $f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k$.
$\sigma_{i,j}$	KZG homomorphic tag for $f_{i,j}(X)$: $\sigma_{i,j} = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$.
$L_{i,j}$	Merkle leaf for tag authentication: $L_{i,j} = H_1(\text{leaf} \text{fid}_i j \text{enc}(\sigma_{i,j}))$.
Γ_i	Root hash of the Merkle tree for file i .
J	Challenged chunk index set, $J \subseteq [n]$ with $ J = c$.
$v_{i,j}$	Random coefficient for challenged chunk j of file i (written as v_j when the same coefficient notation is used for all files).
r_i	Random evaluation point for file i (used for per-file quotient construction).
$f_i(X)$	Aggregated polynomial for file i : $f_i(X) = \sum_{j \in J} v_{i,j} f_{i,j}(X)$.
y_i	Evaluation claim for file i : $y_i = f_i(r_i)$.
b_i	Commitment to the aggregated file polynomial: $b_i = g^{f_i(\tau)} = \prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$. In ProofGen, it is computed from the coefficients of $f_i(X)$ using the SRS.
μ_i	Fresh additive mask for the per-file evaluation y_i .
$\tilde{y}_i$	Masked evaluation: $\tilde{y}_i = y_i + \mu_i$.
R_i, Π_R	Single-base mask commitment $R_i = g^{\mu_i}$ and Schnorr proof of knowledge of μ_i .
$w_i(X)$	Per-file quotient polynomial: $w_i(X) = (f_i(X) - y_i)/(X - r_i)$.
C_{w_i}	Commitment to the per-file quotient witness: $C_{w_i} = g^{w_i(\tau)}$.
ρ	Fiat-Shamir derived folding scalar for batch quotient verification (sampled after $\{C_{w_i}\}$ and Π_R are fixed).
L, R	Folded verification accumulators checked by $e(L, h) = e(R, h^\tau)$.
chal	Challenge tuple $(J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$.
π	Complete audit proof $(\{\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, b_i, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, \Pi_R)$.

Lemma 2 (Random Linear Folding). *Let $\Delta_1, \dots, \Delta_m \in \mathbb{F}_p$ be field elements that are fixed before ρ is sampled. If at least one Δ_i is non-zero and $\rho \leftarrow \mathbb{F}_p$ is uniform, then*

$$\Pr_\rho \left[\sum_{i=1}^m \rho^{i-1} \Delta_i = 0 \right] \leq \frac{m-1}{p}.$$

Proof. The folded error polynomial $P_\Delta(T) = \sum_{i=1}^m \Delta_i T^{i-1}$ is non-zero and has degree at most $m-1$. The claim follows immediately from Lemma 1. $\square$

C. KZG Polynomial Commitment

We use the Kate–Zaverucha–Goldberg (KZG) polynomial commitment scheme [2] over bilinear groups $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ of prime order p . The pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is efficiently computable, bilinear, and non-degenerate. Let $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$ be generators.

a) *Setup.*: A setup algorithm samples a trapdoor $\tau \leftarrow \mathbb{F}_p$ and publishes the KZG part of the SRS supporting degree- $< s$ polynomials:

$$\text{SRS} = (g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau).$$

The trapdoor τ must remain hidden from any prover after setup.

b) *Commitment.*: For $f(X) = \sum_{k=0}^{s-1} a_k X^k \in \mathbb{F}_p^{<s}[X]$, the KZG commitment is

$$C_f := g^{f(\tau)} = \prod_{k=0}^{s-1} (g^{\tau^k})^{a_k} \in \mathbb{G}_1. \quad (1)$$

The commitment is additively homomorphic in the exponent: for commitments C_f and C_q and scalars $\alpha, \beta \in \mathbb{F}_p$, one has $C_f^\alpha C_q^\beta = g^{(\alpha f + \beta q)(\tau)}$.

c) *Opening and Verification.*: To open C_f at point $r \in \mathbb{F}_p$ with value $y = f(r)$, the prover computes

$$w(X) = \frac{f(X) - y}{X - r}, \quad \pi = g^{w(\tau)}.$$

The verifier accepts if

$$e(C_f g^{-y}, h) \stackrel{?}{=} e(\pi, h^\tau h^{-r}). \quad (2)$$

Equation (2) is equivalent, in the exponent, to $f(\tau) - y = (\tau - r)w(\tau)$. Under the standard q -SDH assumption for the SRS degree bound, KZG commitments are computationally binding: a PPT adversary cannot open the same commitment at the same point to two distinct values, nor open it to a value inconsistent with the committed polynomial, except with negligible probability.

D. Multi-Polynomial Batch Evaluation (MPBE)

Batch verification in FoldAudit is based on random linear folding of multiple KZG quotient equations. Suppose the verifier must check m claims $f_i(r_i) = y_i$ with commitments $b_i = g^{f_i(\tau)}$ and quotient commitments $C_{w_i} = g^{w_i(\tau)}$. The individual KZG relation for file i can be written as

$$e(b_i g^{-y_i} C_{w_i}^{r_i}, h) \stackrel{?}{=} e(C_{w_i}, h^\tau). \quad (3)$$

Indeed, if $w_i(X) = (f_i(X) - y_i)/(X - r_i)$, then $b_i g^{-y_i} C_{w_i}^{r_i} = g^{\tau w_i(\tau)}$.

Verifying all equations in (3) separately would require $O(m)$ pairings. Instead, after all commitments are fixed, the verifier samples a Fiat–Shamir folding challenge $\rho \in \mathbb{F}_p$ and checks a single folded equation:

$$e\left(\prod_{i=1}^m (b_i g^{-y_i} C_{w_i}^{r_i})^{\rho^{i-1}}, h\right) \stackrel{?}{=} e\left(\prod_{i=1}^m C_{w_i}^{\rho^{i-1}}, h^\tau\right). \quad (4)$$

If any individual equation has a non-zero exponent defect Δ_i , then the folded equation can still hold only when $\sum_i \rho^{i-1} \Delta_i = 0$, which occurs with probability at most $(m-1)/p$ by Lemma 2. This is the algebraic source of the batch soundness bound used in Theorem 2.

FoldAudit masks each value y_i as

$$\tilde{y}_i = y_i + \mu_i, \quad R_i = g^{\mu_i},$$

where $\mu_i \leftarrow \mathbb{F}_p$ is freshly sampled in every audit round. The verifier never receives y_i directly; instead, it substitutes $R_i g^{-\tilde{y}_i} = g^{-y_i}$ inside the KZG opening equation. Thus the masked per-file relation becomes

$$e(b_i R_i g^{-\tilde{y}_i} C_{w_i}^{r_i}, h) \stackrel{?}{=} e(C_{w_i}, h^\tau). \quad (5)$$

To make the mask witness extractable in the soundness proof, FoldAudit attaches a Fiat–Shamir Schnorr proof of knowledge for every R_i . Let

$$\text{stmt}_R = (\text{chal}, \{\Gamma_i\}, \{b_i\}, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}).$$

For each i , the prover samples $a_i \leftarrow \mathbb{F}_p$, computes $A_i = g^{a_i}$, derives

$$c_i = H_2(\text{mask-pok}, \text{stmt}_R, i, A_i),$$

and returns $z_i = a_i + c_i \mu_i$. The Schnorr proof is

$$\Pi_R = \{(A_i, z_i)\}_{i=1}^m,$$

and verification checks $g^{z_i} = A_i R_i^{c_i}$ for every i . By special soundness of Schnorr proofs and the Fiat–Shamir transform in the random oracle model, an accepting proof yields an extractable witness μ_i satisfying $R_i = g^{\mu_i}$.

After all proof components, including Π_R , are fixed, the verifier derives ρ and checks the folded masked equation

$$e\left(\prod_{i=1}^m (b_i R_i g^{-\tilde{y}_i} C_{w_i}^{r_i})^{\rho^{i-1}}, h\right) \stackrel{?}{=} e\left(\prod_{i=1}^m C_{w_i}^{\rho^{i-1}}, h^\tau\right). \quad (6)$$

This restores the basic additive-mask version $\tilde{y}_i \leftarrow y_i + \mu_i$. Since $R_i = g^{\mu_i}$ is public and single-base, the masking is not statistically hiding for low-entropy y_i : a verifier that can enumerate candidates may test $R_i \stackrel{?}{=} g^{\tilde{y}_i - y'}$. The privacy statement for this version therefore requires challenged evaluations to have sufficient min-entropy against feasible enumeration.

E. Merkle Tree

A Merkle tree is a hash-based vector commitment that supports succinct membership authentication. Let $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be a collision-resistant hash function. FoldAudit uses domain-separated leaves that bind each authenticated tag to its file identifier and chunk index:

$$L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j})), \quad j \in [n],$$

where fid_i is a canonical file identifier and $\text{enc}(\cdot)$ is an injective group-element encoding. The domain label, file identifier, and index prevent cross-file reuse and position-swapping attacks even when identical tags appear at different positions. Internal nodes are computed recursively with a separate domain label as $N = H_1(\text{node} \parallel N_{\text{left}} \parallel N_{\text{right}})$ until a root Γ_i is obtained. A membership proof for $\sigma_{i,j}$ consists of the sibling nodes on the path from $L_{i,j}$ to Γ_i . Verification recomputes the root from $(\text{fid}_i, j, \sigma_{i,j})$ and the path, and accepts only if it equals the trusted root. For a balanced binary tree, each authentication path contains $\lceil \log_2 n \rceil$ hash digests.

III. SYSTEM MODEL AND SECURITY MODEL

A. System Model

a) *System Entities*: We consider a batch-verifiable Provable Data Possession (PDP) architecture comprising three entities: Data Owner, a Cloud Server and Verifier.

- **Data Owner (DO)**: The DO outsources a batch of m files to the CS for long-term storage. To ensure data integrity without maintaining local copies, the DO delegates periodic auditing to the V.
- **Cloud Server (CS)**: The CS is modeled as untrusted and computationally bounded: it may delete, corrupt, or selectively withhold files or chunks to save storage or bandwidth.
- **Verifier (V)**: The V is honest-but-curious. It executes the auditing protocol faithfully but may attempt to infer sensitive information from audit transcripts.

b) *File Representation*: A file is divided into multiple chunks, and each **chunk** can be represented through polynomial coefficients. Each coefficient is referred to as a **sector**. Let m denote the number of files in a single audit batch, n the number of chunks per file, s the number of sectors per chunk. Namely, each chunk is parsed into s field elements from $\mathbb{F}_p$ and algebraically encoded as a polynomial of degree at most $s-1$:

$$f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k, \quad i \in [m], j \in [n],$$

where $m_{i,j,k}$ represents the k -th sector of the j -th chunk in file i . A **tag** is used to represent the j -th chunk through a KZG polynomial commitment. i.e., $\sigma_{i,j} = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$ by Equation (1). Further, tags of each file are organized into a Merkle Tree to guarantee correct indexing and membership.

c) *Protocol Workflow*: The provable data possession scheme for multiple files operates through five polynomial-time algorithms:

- $\text{Setup}(\lambda) \rightarrow \text{pp}$: Generates public parameters pp , including bilinear group tuples $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, generators (g, h) , a structured reference string (SRS) for degree $s-1$, and cryptographic hash functions H_1 (for Merkle Tree construction) and H_2 (for Fiat-Shamir transcript hashing).
- $\text{Store}(\{F_i\}_{i=1}^m) \rightarrow (\text{data}, \text{meta})$: The DO encodes each chunk into $f_{i,j}(X)$, computes tags $\sigma_{i,j}$, constructs the indexed Merkle Tree over $L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$ to obtain the root hash Γ_i for each file, and outsources $\{F_i, \{\sigma_{i,j}\}, \text{Merkle trees}\}$ to the CS. The DO securely records $\{\Gamma_i\}_{i=1}^m$ and shares them with V.
- $\text{Challenge}(m, n, c) \rightarrow \text{chal}$: The verifier samples a shared challenge index set $J \subseteq [n]$ with $|J| = c$, random aggregation coefficients $\{v_{i,j}\}_{i \in [m], j \in J} \leftarrow \mathbb{F}_p$, and per-file evaluation points $\{r_i\}_{i=1}^m \leftarrow \mathbb{F}_p$. The challenge is $\text{chal} = (J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}, \text{nonce})$, where nonce is a random value to resist replay attack.
- $\text{ProofGen}(\text{chal}, \text{data}, \text{meta}) \rightarrow \pi$: The CS aggregates challenged chunks J and outputs the corresponding non-interactive proofs π .

- $\text{Verify}(\text{chal}, \pi, \{\Gamma_i\}, \text{pp}) \rightarrow \{accept, reject\}$: V validates the integrity of all files $[m]$ via the proofs π .

B. Security Model

The security of the proposed batch-PDP scheme is formalized through three core properties: correctness, soundness, and privacy.

Correctness: If all parties (DO, CS, and V) execute the protocol honestly according to the specified algorithms, the Verify algorithm outputs *accept* with probability 1.

Soundness: Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary controlling the CS. $\mathcal{A}$ wins the soundness game if it can produce a proof π that causes Verify to output *accept* for a challenge involving corrupted, modified, or unavailable data chunks with non-negligible probability.

Privacy against Honest-but-Curious Verifier: Given the public audit transcript and the generated proof π , a PPT verifier cannot recover challenged evaluations $\{y_i = f_i(r_i)\}_{i=1}^m$ beyond feasible candidate testing, provided these evaluations have sufficient min-entropy. Raw data sectors remain hidden under the discrete-log hardness of the published commitments and the secrecy of the KZG trapdoor.

IV. PROPOSED SCHEME

A. Construction

a) *Setup*: The Setup algorithm initializes the cryptographic environment and publishes the system-wide public parameters pp . It first generates a bilinear group tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ of prime order p , selects public generators $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$, and samples a secret trapdoor $\tau \in \mathbb{F}_p$. Using τ , the algorithm constructs the structured reference string (SRS) $(g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$, which supports polynomial commitments of degree strictly less than s . Cryptographic hash functions are fixed: H_1 for Merkle Tree construction, and H_2 for Fiat-Shamir transcript binding. Finally, $\text{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h, \text{SRS}, H_1, H_2)$ is published. Note that each DO has its own trapdoor τ to enable batch of polynomials and guarantee privacy of DO's files.

Algorithm 1 Setup

Require: security parameter λ

Ensure: public parameters pp

- 1: Generate $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h)$ and $\text{SRS} = (g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$.
 - 2: Fix cryptographic hashes H_1, H_2 .
 - 3: Publish $\text{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h, \text{SRS}, H_1, H_2)$.
-

b) *Files Storing*: The Store algorithm handles data preprocessing, cryptographic tagging, and metadata generation for outsourcing. For each file F_i ($i \in [m]$), the Data Owner parses every chunk $j \in [n]$ into s field elements and algebraically encodes them as a polynomial $f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k$. A KZG homomorphic tag $\sigma_{i,j} = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$ is computed for each chunk, providing a compact cryptographic binding to the underlying data sectors. Note that τ is chosen and known by the data owner, so the computation of

tag $\sigma_{i,j}$ can be performed in constant time. Then, each tag is encoded as a domain-separated Merkle leaf $L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$, and the leaves are organized into a Merkle Tree with root Γ_i . The tree roots $\{\Gamma_i\}$ are securely retained by the DO (and later shared with the Verifier), while the raw data and tags are outsourced to the CS. This structure guarantees both data integrity and index authenticity without requiring the DO to maintain local copies.

Algorithm 2 Store

Require: files $\{F_i\}_{i=1}^m$

Ensure: outsourced data, metadata

```

1: for  $i = 1$  to  $m$  do
2:   for  $j = 1$  to  $n$  do
3:     Parse chunk  $j$  into  $s$  sectors and form  $f_{i,j}(X)$ .
4:     Compute tag  $\sigma_{i,j} = g^{f_{i,j}(\tau)}$ .
5:      $L_{i,j} \leftarrow H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$ .
6:   end for
7:   Build the  $i$ th Merkle Tree over leaves  $\{L_{i,j}\}_{j=1}^n$ .
8:   Obtain root  $\Gamma_i$ .
9: end for
10: DO sends  $(F_i, \{\sigma_{i,j}\})$  to CS and records  $\{\Gamma_i\}$  for V.
```

c) **Challenge Generation:** The Chal algorithm initiates the audit by sampling a challenge tuple $\text{chal} = (J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$. An index set $J \subseteq [n]$ of size c is selected and applied to every audited file. Independent coefficients $\{v_{i,j}\}$ are drawn uniformly from $\mathbb{F}_p$ to randomize each file's linear aggregation and prevent targeted corruption. Independent per-file evaluation points $\{r_i\}$ are sampled to support file-specific quotient checks, while a fresh nonce prevents replay and precomputation attacks.

Algorithm 3 Challenge

Require: parameters (m, n, c)

Ensure: challenge chal

```

1: Sample one shared challenged index set  $J \subseteq [n]$ ,  $|J| = c$ .
2: Sample random  $\{v_{i,j} \leftarrow \mathbb{F}_p\}_{i \in [m], j \in J}$ .
3: Sample random  $\{r_i \leftarrow \mathbb{F}_p\}_{i=1}^m$ .
4: Set  $\text{chal} = (J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$ .
```

d) **Proof Generation:** Upon receiving $\text{chal} = (J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$, the CS executes ProofGen independently for each audited file and outputs a fixed proof transcript. The proof contains authenticated challenged tags, one masked evaluation, one single-base mask commitment, one quotient commitment, one aggregated tag per file, and Schnorr proofs of knowledge for the mask exponents.

Algorithm 4 ProofGen

Require: challenge $\text{chal} = (J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$, files $\{F_i\}_{i=1}^m$, tags $\{\sigma_{i,j}\}$, SRS

Ensure: proof π

```

1: for  $i = 1$  to  $m$  do
  Step 1: Per-File Tag Authentication Paths Building
2:   Build Merkle paths  $\text{Auth}_i$  for the leaves  $L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$ ,  $j \in J$ , with respect to  $\Gamma_i$ 
  Step 2: Per-File Polynomial Aggregation and Masked Quotient Commitment
3:    $f_i(X) \leftarrow \sum_{j \in J} v_{i,j} f_{i,j}(X) = \sum_{k=0}^{s-1} a_{i,k} X^k$ 
4:    $b_i \leftarrow g^{f_i(\tau)} = \prod_{k=0}^{s-1} (g^{\tau^k})^{a_{i,k}}$ 
5:    $y_i \leftarrow f_i(r_i)$ 
6:   Sample  $\mu_i \xleftarrow{\$} \mathbb{F}_p$ 
7:    $\tilde{y}_i \leftarrow y_i + \mu_i$ 
8:    $R_i \leftarrow g^{\mu_i}$ 
9:    $w_i(X) \leftarrow \frac{f_i(X) - y_i}{X - r_i} = \sum_{k=0}^{s-2} d_{i,k} X^k \triangleright \text{Polynomial division over } \mathbb{F}_p$ 
10:   $C_{w_i} \leftarrow g^{w_i(\tau)} = \prod_{k=0}^{s-2} (g^{\tau^k})^{d_{i,k}}$ 
11: end for
  Step 3: Schnorr Mask Proofs
12:  $\text{stmt}_R \leftarrow (\text{chal}, \{\Gamma_i\}, \{b_i\}, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\})$ 
13: for  $i = 1$  to  $m$  do
14:   Sample  $a_i \xleftarrow{\$} \mathbb{F}_p$  and set  $A_i \leftarrow g^{a_i}$ 
15:    $c_i \leftarrow H_2(\text{mask-pok}, \text{stmt}_R, i, A_i)$ 
16:    $z_i \leftarrow a_i + c_i \mu_i$ 
17: end for
18:  $\Pi_R \leftarrow \{(A_i, z_i)\}_{i=1}^m$ 
  Step 4: Output Proof
19: return
```

$$\pi = (\{\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, b_i, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, \Pi_R).$$

- 1) *Step 1: Per-File Tag Authentication Paths Building.* For each file $i \in [m]$, the CS collects the challenged tags $\{\sigma_{i,j}\}_{j \in J}$ and builds Merkle authentication paths for the bound leaves $L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$ with respect to the root Γ_i .
- 2) *Step 2: Per-File Aggregation and Masked Evaluation.* The CS aggregates the challenged chunks into $f_i(X) = \sum_{j \in J} v_{i,j} f_{i,j}(X)$ and computes the aggregate commitment directly from the coefficient vector of $f_i(X)$: $b_i = g^{f_i(\tau)} = \prod_{k=0}^{s-1} (g^{\tau^k})^{a_{i,k}}$, where $f_i(X) = \sum_{k=0}^{s-1} a_{i,k} X^k$. This avoids exponentiating the c challenged tags in proof generation; the equality $b_i = \prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$ is enforced later by the verifier. It then evaluates $y_i = f_i(r_i)$, samples fresh $\mu_i \xleftarrow{\$} \mathbb{F}_p$, outputs the masked value $\tilde{y}_i = y_i + \mu_i$, and commits to the mask as $R_i = g^{\mu_i}$.
- 3) *Step 3: Per-File Quotient Witnesses.* Since $y_i = f_i(r_i)$, the polynomial $f_i(X) - y_i$ is divisible by $X - r_i$. The CS computes $w_i(X) = \frac{f_i(X) - y_i}{X - r_i}$ with $\deg(w_i) \leq s - 2$ and publishes the quotient commitment $C_{w_i} = g^{w_i(\tau)}$. The CS also generates $\Pi_R = \{(A_i, z_i)\}_{i=1}^m$, a set of Fiat-Shamir Schnorr proofs proving knowledge of μ_i for each $R_i = g^{\mu_i}$. The Fiat-Shamir scalar ρ is derived after all

Algorithm 5 Verify

Require: challenge chal = $(J, \{v_{i,j}\}_{i \in [m], j \in J}, \{r_i\}_{i=1}^m, \text{nonce}), \text{ proof } \pi, \text{ Merkle roots } \{\Gamma_i\}_{i=1}^m, \text{ public parameters pp}$

Ensure: accept or reject

1: Parse π as

$$\pi = (\{\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, b_i, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, \Pi_R).$$

Step 1: Tag Authentication

2: **for** $i = 1$ to m **do**

3: Verify Auth_i against Γ_i for the leaves $L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$, $j \in J$

4: **if** verification fails **then**

5: **return reject**

6: **end if**

7: **end for**

Step 2: Per-File Tag Consistency

8: **for** $i = 1$ to m **do**

9: $\hat{b}_i \leftarrow \prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$

10: **if** $b_i \neq \hat{b}_i$ **then**

11: **return reject**

12: **end if**

13: **end for**

Step 3: Schnorr Mask Proof Verification

14: Parse Π_R as $\{(A_i, z_i)\}_{i=1}^m$

15: $\text{stmt}_R \leftarrow (\text{chal}, \{\Gamma_i\}, \{b_i\}, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\})$

16: **for** $i = 1$ to m **do**

17: $c_i \leftarrow H_2(\text{mask-pok}, \text{stmt}_R, i, A_i)$

18: **if** $g^{z_i} \neq A_i R_i^{c_i}$ **then**

19: **return reject**

20: **end if**

21: **end for**

Step 4: Fiat-Shamir Folding Challenge

22: $\rho \leftarrow H_2(\text{fold}, \text{chal}, \{\Gamma_i\}_{i=1}^m, \{b_i\}_{i=1}^m, \{\tilde{y}_i\}_{i=1}^m, \{R_i\}_{i=1}^m, \{C_{w_i}\}_{i=1}^m, \Pi_R)$

Step 5: Folded Masked KZG Verification

23: $L \leftarrow \prod_{i=1}^m (b_i R_i g^{-\tilde{y}_i} C_{w_i}^{r_i})^{\rho^{i-1}}$

24: $R \leftarrow \prod_{i=1}^m C_{w_i}^{\rho^{i-1}}$

25: **if** $e(L, h) \neq e(R, h^\tau)$ **then**

26: **return reject**

27: **end if**

28: **return accept**

(C_{w_i}, Π_R) are fixed and is used in the verifier's folded pairing check.

4) *Step 4: Output.* The proof is exactly $\pi = (\{\text{Auth}_i, \tilde{y}_i, R_i, C_{w_i}, b_i, \{\sigma_{i,j}\}_{j \in J}\}_{i=1}^m, \Pi_R)$.

e) *Verification:* The Verify algorithm validates index authenticity, tag aggregation, transcript binding, and quotient consistency. The quotient consistency check is performed by one folded pairing equation over the masked KZG openings.

1) *Step 1: Tag Authentication.* For every file, the verifier checks Auth_i against the trusted Merkle root Γ_i for the challenged leaves $L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$. This prevents the CS from substituting tags across files or chunk positions.

2) *Step 2: Per-File Tag Consistency.* The verifier recomputes

$\hat{b}_i = \prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$ and rejects unless $\hat{b}_i = b_i$ for every $i \in [m]$. Hence each b_i is forced to be the homomorphic commitment to the challenged aggregate polynomial $f_i(X)$.

3) *Step 3: Schnorr Mask Proof Verification.* The verifier checks Π_R by recomputing $c_i = H_2(\text{mask-pok}, \text{stmt}_R, i, A_i)$ and accepting only if $g^{z_i} = A_i R_i^{c_i}$ for every file. This makes the mask exponent μ_i extractable from any accepting transcript in the random oracle model.

4) *Step 4: Fiat-Shamir Folding Challenge.* The verifier derives $\rho = H_2(\text{fold}, \text{chal}, \{\Gamma_i\}, \{b_i\}, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}, \Pi_R)$ after all public proof components have been parsed. Since ρ is bound to the fixed transcript, the prover cannot adapt the quotient witnesses to the folding coefficient.

5) *Step 5: Folded Masked KZG Verification.* The verifier checks $e(L, h) = e(R, h^\tau)$ for $L = \prod_i (b_i R_i g^{-\tilde{y}_i} C_{w_i}^{r_i})^{\rho^{i-1}}$ and $R = \prod_i C_{w_i}^{\rho^{i-1}}$. Since $R_i = g^{\mu_i}$ in honest proofs, the term $R_i g^{-\tilde{y}_i}$ equals g^{-y_i} , so the check is exactly the folded KZG opening relation for the hidden values $y_i = \tilde{y}_i - \mu_i$.

If all checks pass, the verifier outputs *accept*; otherwise, it outputs *reject*.

B. Security Analysis

We establish correctness, soundness, and privacy for the protocol described in Algorithms 2–5. All adversaries considered below are probabilistic polynomial-time (PPT) algorithms in the security parameter λ . A function $\varepsilon(\lambda)$ is negligible if it is smaller than $1/\lambda^c$ for every constant $c > 0$ and all sufficiently large λ . The analysis relies on the following assumptions.

Assumption 1 (Collision Resistance of H_1). *The Merkle-tree hash function $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is collision-resistant. Namely, for every PPT adversary $\mathcal{A}$,*

$$\Pr[x \neq x' \wedge H_1(x) = H_1(x') : (x, x') \leftarrow \mathcal{A}(1^\lambda)] \leq \text{negl}(\lambda).$$

Consequently, a PPT adversary cannot produce two distinct tag-index sets that authenticate to the same Merkle root except with negligible probability.

Assumption 2 (Fiat-Shamir Random Oracle). *The transcript hash H_2 is modeled as a programmable random oracle with domain separation for mask proofs and proof folding. In particular, once the tuple $(\text{chal}, \{\Gamma_i\}, \{b_i\}, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}, \Pi_R)$ is fixed, the value*

$$\rho = H_2(\text{fold}, \text{chal}, \{\Gamma_i\}, \{b_i\}, \{\tilde{y}_i\}, \{R_i\}, \{C_{w_i}\}, \Pi_R)$$

is distributed uniformly over $\mathbb{F}_p$ from the adversary's perspective, except with negligible probability.

Assumption 3 (Schnorr Mask-PoK Knowledge Soundness). *The Fiat-Shamir Schnorr proof Π_R is knowledge-sound in the random oracle model: any PPT prover that makes all checks $g^{z_i} = A_i R_i^{c_i}$ accept admits an extractor that outputs $\{\mu_i\}_{i=1}^m$ such that $R_i = g^{\mu_i}$ for all i , except with negligible probability.*

Assumption 4 (KZG Binding and Knowledge Soundness). *For the SRS degree bound $q \geq s - 1$, the q -SDH assumption holds in the bilinear group tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$. Equivalently for our use, the KZG scheme instantiated with the KZG SRS $(g, g^\tau, \dots, g^{\tau^q}, h, h^\tau)$ is computationally evaluation-binding and knowledge-sound: for every PPT adversary that outputs (C, r, y, π) satisfying the KZG verification equation, there exists an extractor that recovers a degree- $\leq q$ polynomial $f(X)$ such that $C = g^{f(\tau)}$ and $f(r) = y$, except with negligible probability. In particular, no PPT adversary can open a fixed commitment at a fixed point to an incorrect or second distinct value with non-negligible probability.*

Assumption 5 (Discrete Logarithm Hardness). *The discrete logarithm problem is hard in $\mathbb{G}_1$. That is, for every PPT adversary $\mathcal{A}$,*

$$\Pr[y' = y : y \leftarrow \mathbb{F}_p, Y = g^y, y' \leftarrow \mathcal{A}(g, Y)] \leq \text{negl}(\lambda).$$

This assumption is used for the standard hardness of extracting committed exponents from single-base group commitments.

Assumption 6 (Evaluation Min-Entropy). *For every honest audit instance, each challenged evaluation $y_i = f_i(r_i)$ has conditional min-entropy at least κ from the verifier's perspective before observing $(\tilde{y}_i, R_i)$. A verifier bounded to q candidate tests therefore guesses y_i with probability at most $q2^{-\kappa}$.*

Theorem 1 (Correctness). *If the Data Owner, Cloud Server, and Verifier execute Algorithms 2, 3, 4, and 5 honestly, then Verify outputs accept with probability 1.*

Proof. For each file $i \in [m]$, the honest CS computes $f_i(X) = \sum_{j \in J} v_{i,j} f_{i,j}(X)$ and $y_i = f_i(r_i)$. Hence $X - r_i$ divides $f_i(X) - y_i$, and $w_i(X) = \frac{f_i(X) - y_i}{X - r_i}$ is a polynomial of degree at most $s - 2$.

The Merkle paths Auth_i are generated from the challenged bound leaves $L_{i,j} = H_1(\text{leaf} \parallel \text{fid}_i \parallel j \parallel \text{enc}(\sigma_{i,j}))$, so Step 1 of Verify accepts. For Step 2,

$$\hat{b}_i = \prod_{j \in J} \sigma_{i,j}^{v_{i,j}} = \prod_{j \in J} \left(g^{f_{i,j}(\tau)} \right)^{v_{i,j}} = g^{f_i(\tau)} = b_i.$$

The mask and quotient commitments satisfy $R_i = g^{\mu_i}$ and $C_{w_i} = g^{w_i(\tau)}$ by construction. The Schnorr equations also accept because $z_i = a_i + c_i \mu_i$ implies $g^{z_i} = g^{a_i} (g^{\mu_i})^{c_i} = A_i R_i^{c_i}$. For the folded masked relation, the honest witnesses satisfy

$$\begin{aligned} b_i g^{\mu_i} g^{-\tilde{y}_i} C_{w_i}^{r_i} &= g^{f_i(\tau)} g^{\mu_i} g^{-(y_i + \mu_i)} g^{r_i w_i(\tau)} \\ &= g^{f_i(\tau) - y_i + r_i w_i(\tau)} = g^{\tau w_i(\tau)}. \end{aligned}$$

Therefore each individual masked KZG equation holds, and their Fiat-Shamir folded equation in Algorithm 5 also holds by bilinearity. Thus every verification step accepts. $\square$

Theorem 2 (Soundness). *Let $\mathcal{A}$ be a PPT adversary controlling the CS. Under Assumptions 1, 2, 3, and 4, if a challenge touches corrupted, modified, or unavailable chunks, then $\mathcal{A}$*

causes Verify to accept without possessing the challenged aggregate data with probability at most

$$\frac{m-1}{p} + \text{negl}(\lambda).$$

Proof. Merkle verification binds each challenged tag $\sigma_{i,j}$ to $(\text{fid}_i, j, \Gamma_i)$ except with negligible probability under Assumption 1. The tag-consistency step then forces $b_i = \prod_{j \in J} \sigma_{i,j}^{v_{i,j}} = g^{f_i(\tau)}$ for the authenticated aggregate polynomial $f_i(X)$.

By Assumption 3, every accepting Π_R yields extractable witnesses $\{\mu_i\}_{i=1}^m$ satisfying $R_i = g^{\mu_i}$. Write $C_{w_i} = g^{w_i}$ for the corresponding quotient-commitment exponent and define $y_i = \tilde{y}_i - \mu_i$. Since (R_i, C_{w_i}, Π_R) is fixed before ρ is sampled, the mask and quotient commitments are fixed before ρ except with negligible probability under Assumption 2. If the folded masked KZG check accepts, then the pairing equation enforces

$$\sum_{i=1}^m \rho^{i-1} (f_i(\tau) - y_i - (\tau - r_i) \omega_i) = 0.$$

For each file define the opening defect

$$\Delta_i = f_i(\tau) - y_i - (\tau - r_i) \omega_i.$$

If every $\Delta_i = 0$, then each C_{w_i} is a valid KZG witness opening b_i at r_i to y_i . Producing such witnesses for corrupted or unavailable challenged data without the corresponding aggregate polynomial violates KZG knowledge soundness except with negligible probability.

Otherwise, at least one Δ_i is nonzero. By Assumption 2, ρ is uniform after all values $(b_i, \tilde{y}_i, R_i, C_{w_i}, \Pi_R)$ are fixed, so the adversary must make the nonzero polynomial

$$P_\Delta(\rho) = \sum_{i=1}^m \Delta_i \rho^{i-1}$$

vanish at a random field point. This polynomial has degree at most $m - 1$, so by Lemma 1, $\Pr[P_\Delta(\rho) = 0] \leq (m - 1)/p$. Combining the negligible tag/KZG failures with the folding error gives the claimed bound. $\square$

Theorem 3 (Privacy against Honest-but-Curious Verifier). *Under Assumptions 2, 5, and 6, if every mask μ_i is freshly sampled from $\mathbb{F}_p$, then an honest-but-curious verifier that performs at most q candidate tests recovers a challenged evaluation $y_i = f_i(r_i)$ from $(\tilde{y}_i, R_i)$ with probability at most $q2^{-\kappa} + \text{negl}(\lambda)$. Recovering raw sectors $\{m_{i,j,k}\}$ from the remaining group commitments requires solving discrete logarithms or learning the trapdoor τ .*

Proof. The field element $\tilde{y}_i = y_i + \mu_i$ is uniform over $\mathbb{F}_p$ when considered alone because μ_i is uniform and used once. The Schnorr proof Π_R is honest-verifier zero-knowledge under Fiat-Shamir and therefore does not reveal more about μ_i than the public commitment R_i . In the rollback version, however, the transcript contains $R_i = g^{\mu_i}$. Therefore a verifier can test a guessed value y' by checking whether $R_i = g^{\tilde{y}_i - y'}$. The protection of y_i is consequently computational rather than statistical: by Assumption 6, a verifier limited to q guesses succeeds through this candidate test with probability at most $q2^{-\kappa}$.

For values outside the verifier's tested candidate set, recovering μ_i from $R_i = g^{\mu_i}$ requires solving a discrete logarithm in $\mathbb{G}_1$. Likewise, recovering raw sector coefficients from $\sigma_{i,j}$, b_i , or C_{w_i} would require solving discrete logarithms or learning the trapdoor τ . Thus the transcript hides challenged evaluations only under the stated min-entropy condition and hides raw sectors under the discrete-log/KZG setup assumptions. $\square$

C. Complexity Analysis

We measure computational overhead in terms of group exponentiations (E) and bilinear pairings (P). Field arithmetic, hash evaluations, Fiat–Shamir transcript hashing, group multiplications, and Merkle-path hashing are omitted from the group-operation count. Communication overhead is measured in bits using $|p|$, $|\mathbb{G}_1|$, and $|h|$ for a field element, a $\mathbb{G}_1$ element, and a hash digest, respectively.

a) Computation Complexity.

- **Store Algorithm.** Since the DO holds the trapdoor τ , each tag $\sigma_{i,j} = g^{f_{i,j}(\tau)}$ costs one exponentiation after evaluating $f_{i,j}(\tau)$ in the field. Across m files and n chunks, the storage-side group cost is

$$mnE.$$

- **ProofGen Algorithm.** For each file, after forming the aggregated polynomial $f_i(X) = \sum_{k=0}^{s-1} a_{i,k} X^k$ in the field, the CS computes the aggregate commitment as $b_i = g^{f_i(\tau)} = \prod_{k=0}^{s-1} (g^{\tau^k})^{a_{i,k}}$ using the SRS. This costs sE , rather than cE , and is equivalent to $\prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$ by the homomorphism of KZG commitments. The CS also computes the mask commitment $R_i = g^{\mu_i}$ using E , the quotient commitment $C_{w_i} = g^{w_i(\tau)}$ using $(s-1)E$, and the Schnorr commitment $A_i = g^{a_i}$ using E . Thus proof generation costs

$$\underbrace{msE}_{\{b_i\}} + \underbrace{mE}_{\{R_i\}} + \underbrace{m(s-1)E}_{\{C_{w_i}\}} + \underbrace{mE}_{\{A_i\}} = (2ms + m)E.$$

Since the audit challenge size satisfies $c > s$ in the intended parameter regime, computing b_i from the coefficient vector of f_i keeps proof generation at $O(msE)$ rather than $O(mcE)$.

- **Verify Algorithm.** The verifier recomputes $\hat{b}_i = \prod_{j \in J} \sigma_{i,j}^{v_{i,j}}$ for all files using mcE . It verifies the Schnorr mask proofs using $2mE$, constructs the folded accumulators $L = \prod_i (b_i R_i g^{-\tilde{y}_i} C_{w_i}^{r_i})^{\rho^{i-1}}$ and $R = \prod_i C_{w_i}^{\rho^{i-1}}$ using $(4m+1)E$ with straightforward multi-exponentiation accounting, and checks one two-pairing equation. Therefore verification costs

$$(mc + 6m + 1)E + 2P.$$

b) Communication Complexity.

- **Store Algorithm.** The DO transmits raw file data to the CS, incurring $mns|p|$ bits. Concurrently, mn homomorphic tags $\{\sigma_{i,j}\}$ are outsourced to enable future homomorphic aggregation, costing $mn|\mathbb{G}_1|$ bits. The DO securely shares m Merkle root hashes $\{\Gamma_i\}_{i=1}^m$ with the verifier for audit authentication, requiring $m|h|$ bits. The

total one-time outsourcing communication is $mns|p| + mn|\mathbb{G}_1| + m|h|$.

$$\underbrace{mns|p|}_{\text{File data}} + \underbrace{mn|\mathbb{G}_1|}_{\text{Tags}} + \underbrace{m|h|}_{\text{Merkle roots}} = mns|p| + mn|\mathbb{G}_1| + m|h|.$$

- **ProofGen Algorithm.** For each file $i \in [m]$, the proof contains c challenged tags $\{\sigma_{i,j}\}_{j \in J}$, three group elements (R_i, C_{w_i}, b_i) , one field element $\tilde{y}_i$, one Schnorr commitment A_i , one Schnorr response z_i , and c Merkle authentication paths. Therefore the total proof communication is

$$m(c+4)|\mathbb{G}_1| + 2m|p| + mc \log_2 n |h|.$$

D. Dynamic Audit-Set Management

The scheme natively supports file addition and removal from the active audit set through independent per-file KZG commitments and modular Merkle trees. Crucially, ProofGen and Verify are stateless per audit round; they operate on the current active file set without maintaining persistent or incremental proof state. Dynamic operations thus only affect the Store phase and metadata synchronization.

When the DO uploads a new file F_{m+1} , it encodes each chunk into $f_{m+1,j}(X)$, computes tags $\sigma_{m+1,j} = g^{f_{m+1,j}(\tau)}$, and constructs a fresh Merkle tree yielding root Γ_{m+1} . The data and tags are outsourced to the CS, while Γ_{m+1} is securely shared with the verifier. The active batch size increments to $m \leftarrow m+1$. Subsequent audits automatically incorporate F_{m+1} via the expanded index collection and Merkle root set.

To remove $F_{i_{\text{del}}}$ from subsequent audits, the DO and V revoke $\Gamma_{i_{\text{del}}}$ from the active metadata set and exclude the corresponding file identifier from future challenges. The CS may reclaim the associated data, tags, and Merkle nodes according to the storage policy, but this protocol action is only an audit-set update and does not prove physical erasure. Surviving files simply skip the removed identifier in subsequent audit loops, incurring zero recomputation of tags or Merkle roots. The active batch size decreases by one, and future ProofGen executions simply iterate over the reduced active set.

Complexity of Dynamic Operations. Adding a single file requires $O(nE)$ time for tag generation, as implied by the Store algorithm. Removing a file from the active audit set only updates the metadata registry and requires no tag generation, Merkle reconstruction for surviving files, SRS regeneration, trapdoor rotation, or global commitment recomputation. Since proof generation is challenge-driven and stateless, dynamic updates incur overhead strictly at the storage and metadata layers. Consequently, the ProofGen and Verify algorithms retain the same complexity as analyzed in Section IV-C.

V. RELATED WORKS

Recent years have witnessed significant advances in provable data possession (PDP) schemes for cloud and decentralized storage systems. The works most closely related to FoldAudit differ in their batching model, privacy treatment, and support for dynamic data.

Xu et al. [8] proposed MPA, a lightweight and updatable auditing scheme that combines polynomial commitments with Merkle authentication for decentralized multi-copy storage. MPA supports full aggregation across files and providers and uses a blockchain-assisted commit-and-reveal procedure to derive unpredictable challenges. Its proof-generation cost is dominated by a degree- $(s-1)$ KZG witness, while the Merkle layer gives efficient block-level updates. However, the multi-file verification equation aggregates all local proofs at a shared evaluation point r . Therefore, the batch check does not provide independent per-file evaluation points or an explicit random-folding soundness bound for cross-file cancellation; it verifies a globally aggregated opening together with Merkle membership paths.

Zhang et al. [6] addressed on-chain overhead for decentralized multi-copy auditing by instantiating the PCS-MPAP batch polynomial commitment. Their batch scheme assigns distinct evaluation points $\{r_l\}_{l=1}^d$ to files and publishes a constant-size per-SSP proof $(\sigma'_i, W_i, W'_i, E_i)$. This construction substantially reduces on-chain storage and avoids linear growth in the number of files. Nevertheless, the batch proof exposes the scalar

$$\begin{aligned} E_i &= \sum_{l=1}^d \beta_l S_{l,i}, \\ S_{l,i} &= F_{l,M_i}(r_l) \\ &= \sum_{(a_j, v_j) \in Q_l} v_j f_{l,m_i,a_j}(r_l), \end{aligned}$$

i.e., an unmasked linear combination of challenged file evaluations. Since the challenge transcript publishes $\{\beta_l, r_l, Q_l\}$, each audit round reveals a public linear equation in the challenged block-polynomial coefficients. Across repeated audits with linearly independent challenge vectors, these equations can be accumulated to recover evaluation values and, eventually, the underlying data-polynomial coefficients for repeatedly challenged blocks. Thus, on a transparent ledger, the PCS-MPAP batch variant of ZhangTPDS23 does not provide evaluation privacy against an honest-but-curious verifier or public observer. The scheme also focuses on multi-copy auditing rather than dynamic file updates.

Yu et al. [5] designed a storage verification scheme for resource-constrained Industrial IoT environments. It combines a homomorphic hash function, polynomial commitments, and a Tag-IMHT authenticated structure to eliminate pairings from verification and to support chunk-level modification, insertion, and deletion. Its privacy-enhanced variant adds a random constant term $\beta\eta$ to the proof polynomial and returns an additional masking tag B . However, the proof still discloses the quotient polynomial $Z_{\text{prf}}(x) = (P_{\text{prf}}(x) - P_{\text{prf}}(z))/(x - z)$. Since the constant mask cancels in this quotient, $Z_{\text{prf}}(x)$ reveals the non-constant coefficients of the challenged aggregate polynomial; repeated audits with different challenge coefficients may therefore leak the underlying data polynomial, while only the constant term is protected by $\beta\eta$. The scheme also does not provide a native multi-file batch verification equation; auditing multiple files requires independent protocol executions.

Miao et al. [7] proposed MKPDP, a blockchain-assisted multi-keyword searchable PDP scheme. Its index matrix and match indexes let the verifier check that all files matching the queried keywords participate in the audit, thereby addressing search-result integrity, pre-matching, and pre-computation attacks. The proof $\{T, \mu_1, R\}$ aggregates authenticators over the matched files and masks the aggregate data value as $\mu_1 = \mu - rh_1(T_w, R)$. Thus, MKPDP supports keyword-filtered multi-file auditing and data privacy, but it is not a polynomial-opening batch protocol and does not provide independent per-file evaluation points or a tight finite-field folding bound.

Despite these advances, existing schemes leave at least one of the following gaps: no native arbitrary multi-file batch audit, no independent per-file polynomial challenge points, no explicit tight batch soundness bound, limited evaluation privacy on transparent ledgers, or dynamic support with a different update granularity. FoldAudit addresses these gaps through Fiat-Shamir folding over independently challenged per-file quotient equations, transcript-bound masked evaluations, and stateless file addition and active audit-set removal.

To facilitate a direct comparison, we adopt the unified notation throughout this section: m files in a single audit batch, each comprising n chunks, where each chunk is algebraically encoded as a polynomial of degree $s-1$ (i.e., s sectors per chunk). Let c denote the number of challenged chunks per file. We count only expensive group exponentiations and pairings; field operations, group multiplications, PRF calls, hashes, and Merkle-path hashing are omitted from Table III. For FoldAudit, the table reports the additive-mask version with Schnorr mask PoK in Algorithms 4 and 5. For ZhangTPDS23 and XuTIFS26, which are originally parameterized by the number of replicas/providers, the table reports the normalized single-replica case $N = 1$ so that the comparison focuses on multi-file integrity rather than replication redundancy. For MiaoSCIS2026, m denotes the number of keyword-matched files participating in the audit.

1.1) Computation Complexity of Yu et al. [5]

- **Store Algorithm.** For file i and chunk j , the data owner evaluates the chunk polynomial at the secret point and computes the homomorphic tag as ours. Therefore, the storage computation is mnE .
- **ProofGen Algorithm.** The only counted group exponentiation is the privacy mask $B = g^\beta$. Therefore, over m independent files, the computation is mE .
- **Verify Algorithm.** For each file, the verifier reconstructs H_Z from the s -coefficient quotient polynomial and checks the privacy-enhanced verification equation involving B^η , c challenged tags, H_Z^{-z} , and $\text{HHF}(P_{\text{prf}}(z))$. Following the operation count reported in YuTC25, this costs $(s+c+4)E$ per independent audit. Thus, auditing m files independently costs $m(s+c+4)E$.

1.2) Communication Complexity of Yu et al. [5]

- **Store Algorithm.** The data owner uploads mns field sectors and mn homomorphic tags to the server, and shares one Merkle root per file with the verifier. Hence the one-time storage communication is $mns|p| + mn|G_1| + m|h|$.

Table II: Functional and Security Feature Comparison for Multi-File Integrity Auditing

Scheme	Multi-File Batch	Independent r_i	Tight Batch Bound	Evaluation Privacy	Dynamic Ops.	Cancellation Resistance
XuTIFS26 [8]	✓	✗	✗	✗	✓	✗
ZhangTPDS23 [6]	✓	✓	✗	✗	✗	✓
YuTC25 [5]	✗	—	✗	✗ [†]	✓	—
MiaoSCIS2026 [7]	✓ [‡]	—	✗	✓	✓ [§]	✗
Ours	✓	✓	✓	✓*	✓	✓

Multi-File Batch: native verification of multiple files in one audit.

Tight Batch Bound: explicit finite-field batch soundness error.

Dynamic Ops.: update support; the supported granularity differs across schemes.

Independent r_i : distinct file-specific polynomial evaluation points in one batch equation.

Evaluation Privacy: hiding challenged evaluation values from honest-but-curious verifiers.

Cancellation Resistance: explicit resistance to cross-file algebraic error cancellation in the batch equation.

—: not applicable because the scheme does not use polynomial multi-file batch evaluation. †: YuTC25's privacy-enhanced variant masks only the constant term by $\beta\eta$; its disclosed quotient polynomial can leak non-constant data-polynomial coefficients under repeated audits. ‡: MKPDP audits keyword-matched files in aggregate rather than arbitrary polynomial openings. §: MKPDP supports file add/delete operations; YuTC25 and XuTIFS26 support chunk/block-level updates, while FoldAudit supports stateless file addition and active audit-set removal. *: FoldAudit uses additive one-time masks with Schnorr mask PoK; privacy requires challenged evaluations to have sufficient min-entropy against candidate enumeration.

- **ProofGen Algorithm.** For each file, the response contains the s -coefficient quotient polynomial, one evaluation value, c challenged tags, one mask B , and c Merkle paths of length $\log n$. Across m files, the proof communication is $m(s+1)|p| + m(c+1)|G_1| + mc \log n|h|$.

2.1) Computation Complexity of Zhang et al. [6]

- **Store Algorithm.** For each file block, the tag $\sigma_{i,j} = (H(F_{id}||i||j) \cdot g^{f_{i,j}(\alpha)})^x$ requires one exponentiation for $g^{f_{i,j}(\alpha)}$ and one exponentiation for the outer secret-key power. Across mn tags this gives $2mnE$.
- **ProofGen Algorithm.** For each file, exponentiating the c challenged tags by their audit coefficients costs cE , and applying the file-folding coefficient β_l costs one more exponentiation. Over m files this contributes $mcE + mE$. The two polynomial-commitment proof elements W and W' are obtained from coefficient vectors of length $s-1$, so their counted work is $(2s-2)E$. Therefore, the total cost is $(mc + m + 2s - 2)E$.
- **Verify Algorithm.** The verifier constructs the challenge term ζ by exponentiating c hashed indices per file and then applying m file-folding powers, costing $(mc+m)E$. It also forms $\psi = g^{-E_1} \cdot W_1^{-Z_T(z)}$ and $u \cdot v^{-z}$, which contributes $3E$. The batch equation $e(\sigma', g)e(\psi, v) = e(\zeta, v)e(W', u \cdot v^{-z})$ contains four pairings. Hence, verification costs $(mc + m + 3)E + 4P$.

2.2) Communication Complexity of Zhang et al. [6]

- **Store Algorithm.** The storage phase sends the file copy, containing mns field sectors, and mn tags. The storage communication is $mns|p| + mn|G_1|$.
- **ProofGen Algorithm.** The published proof is $(\sigma', W_1, W'_1, E_1)$, namely three group elements and one field element. Therefore the audit-proof communication is $3|G_1| + |p|$.

3.1) Computation Complexity of Miao et al. [7]

- **Store Algorithm.** In IndexGen, the data owner computes one match index $\Omega_i = (H_{F_i} \cdot H_2(FID_i))^{-x}$ per file, so the counted work is mE . In AuthGen, the data owner creates one authenticator $\sigma_{i,j} = (H_{F_i} \cdot H_2(FID_i) \cdot H_3(au_{i,j}) \cdot u^{m_{i,j}})^x$ for each block. Following the same

convention as the scheme's algorithmic cost model, this contributes one counted group exponentiation per authenticator after the hash and multiplication terms are omitted, giving mnE . Therefore the full storage-side computation is $(m + mn)E$, with no pairing. If the keyword-index module is stripped and only the PDP authentication core is compared, this phase reduces to mnE .

- **ProofGen Algorithm.** After locating the matched set of m files, the server computes $T = \prod_{i \in R} \prod_{j \in Q} \sigma_{i,j}^{e_{i,j}}$, which costs mcE , and generates the privacy element $R_g = g^r$, which costs one more exponentiation. Thus the proof-generation computation is $(mc + 1)E$.
- **Verify Algorithm.** The verifier exponentiates one match index Ω_i for each matched file and c hash terms inside each matched file, and it also computes u^{μ_1} and $R_g^{h_1(T_w, R_g)}$. These operations contribute $(m + mc + 2)E$. The final verification equation has two pairings, so the verifier computation is $(m(c + 1) + 2)E + 2P$.

3.2) Communication Complexity of Miao et al. [7]

- **Store Algorithm.** The data and authenticators sent to the cloud contribute $mn|p| + mn|G_1|$. The match indexes contain one group element per file, adding $m|G_1|$. Thus the normalized storage communication is $mn|p| + (mn + m)|G_1|$, excluding auxiliary searchable index metadata such as addresses and encrypted index matrices.
- **ProofGen Algorithm.** The audit proof (T, μ_1, R_g) consists of two group elements and one field element, so the proof communication is $|p| + 2|G_1|$.

4.1) Computation Complexity of Xu et al. [8]

- **Store Algorithm.** For each file and block, the data owner forms the block polynomial and computes the tag $\sigma_{i,j} = g^{f_{m_{i,j}}(\alpha)}$. Since polynomial evaluation uses field operations, tag generation contributes one group exponentiation for each of the mn tags. Hashing the Merkle leaves and roots is omitted under the current computation metric. The storage-side TagVerify cost reported by XuTIFS26 is a separate validation phase and is not included in this Store/Setup column. Thus the normalized storage computation is mnE , with no pairing.

- **ProofGen Algorithm.** For each file, the CSP linearly combines the c challenged block polynomials, evaluates the aggregate polynomial at the challenge point, and divides by $X - r$. These are field and polynomial operations. The only counted operation is the KZG witness $w_i = g^{h_i(\alpha)}$. Since $h_i(X)$ has degree at most $s - 2$, computing the witness from the structured public key costs $(s - 1)E$ per local proof. Across m local proofs, proof generation costs $m(s - 1)E$, with no pairing.
- **Verify Algorithm.** The verifier first aggregates the challenged tags by computing $\prod_{j=1}^c \sigma_{i,a_j}^{v_j}$ for every file, which costs mcE . It then computes $g^{val^{global}}$ and $g^{\alpha-r}$, adding $2E$. The accepted batch path checks one global KZG equation, requiring the two pairings $\hat{e}(\sigma^{global}/g^{val^{global}}, g)$ and $\hat{e}(w^{global}, g^{\alpha-r})$. Thus verification costs $(mc + 2)E + 2P$. The fallback diagnosis used only after a failed global check is not included in this normal verification cost.

4.2) Communication Complexity of Xu et al. [8]

- **Store Algorithm.** The storage phase sends mn encrypted blocks, each with s field sectors, and mn polynomial tags. The Merkle root for each file is published as the verification anchor, and the algorithm also supplies authentication paths for stored blocks to the corresponding CSP. Therefore the one-time storage communication is $mns|p| + mn|G_1| + mn(\log_2 n + 1)|h| + m|h|$. If the server reconstructs and stores the Merkle tree locally instead of receiving all paths, the $mn(\log_2 n + 1)|h|$ term can be omitted.
- **ProofGen Algorithm.** For each file, the proof contains one evaluation value val_i , one witness w_i , the c challenged tags, and c Merkle authentication paths. Across m proofs, the audit-proof communication is $m(|p| + (c + 1)|G_1| + c \log_2 n |h|)$.

VI. EVALUATION

Lemma 3 (Overall Acceptance Probability Bound). *Let $\rho \in [0, 1]$ denote the fraction of corrupted or missing chunks in any audited file F_i , and let $d = \lfloor \rho n \rfloor$. When the Cloud Server executes ProofGen over a uniformly sampled challenge set $J \subseteq [n]$ with $|J| = c$, the probability that Verify outputs accept despite the corruption is bounded by:*

$$P_{\text{accept}} \leq \frac{\binom{n-d}{c}}{\binom{n}{c}} + \frac{m-1}{p} + \text{negl}(\lambda) \approx (1-\rho)^c + \text{negl}(\lambda). \quad (7)$$

Proof. Let E_{miss} be the event that the challenge set J contains no corrupted chunks. Since J is sampled uniformly without replacement from $[n]$, the probability of this event is exactly the hypergeometric ratio:

$$\Pr[E_{\text{miss}}] = \frac{\binom{n-d}{c}}{\binom{n}{c}}.$$

When $n \gg c$, this simplifies to $(1 - \rho)^c$. In this scenario, the prover can trivially satisfy all verification checks using only intact data.

Let E_{forge} be the complementary event where J intersects at least one corrupted chunk. For Verify to still accept, the

adversary must successfully forge Merkle authentication paths, satisfy the quotient-consistency pairing, and avoid detection by the Fiat–Shamir folded check. By Theorem 2, under the q -SDH assumption and random oracle model, the probability of bypassing these cryptographic constraints is bounded by $\frac{m-1}{p} + \text{negl}(\lambda)$.

Since E_{miss} and E_{forge} are mutually exclusive, the total false-acceptance probability satisfies $P_{\text{accept}} = \Pr[E_{\text{miss}}] + \Pr[E_{\text{forge}}]$. Substituting the respective bounds yields the stated inequality. For standard cryptographic parameters ($p \approx 2^{256}$ and $m \ll p$), the term $\frac{m-1}{p}$ is negligible, and the audit security is strictly governed by the sampling parameter c . $\square$

Remark 1 (Parameter Selection). *To achieve a target detection rate $1 - \epsilon$ against corruption ratio ρ , the challenge size should satisfy $c \geq \frac{\ln \epsilon}{\ln(1-\rho)}$. For example, guaranteeing 99.9% detection for $\rho \geq 1\%$ requires $c \approx 690$. This bound is independent of the batch size m and applies uniformly across all files in the audit.*

VII. CONCLUSION

REFERENCES

- [1] Luo, Z., Jia, Y., Gracia, A.V.O. and Kate, A. Cauchyproofs: Batch-Updatable Vector Commitment with Easy Aggregation and Application to Stateless Blockchains. In *S&P*, 2025, pp. 1947-1963.
- [2] Kate, A., Zaverucha, G.M. and Goldberg, I. Constant-size commitments to polynomials and their applications. In *Asiacrypt*, 2010, pp. 177-194.
- [3] Boneh, D., Drake, J., Fisch, B. & Gabizon, A. Efficient polynomial commitment schemes for multiple points and polynomials. *Cryptology EPrint Archive*. (2020)
- [4] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, “PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge,” *IACR Cryptology ePrint Archive*, Report 2019/953, 2019. <https://eprint.iacr.org/2019/953>
- [5] H. Yu, H. Zhang, Z. Yang, Y. Chen, and H. Liu, “Efficient and Secure Storage Verification in Cloud-Assisted Industrial IoT Networks,” *IEEE Transactions on Computers*, vol. 74, no. 5, pp. 1702–1716, 2025.
- [6] Q. Zhang, Z. Zhang, J. Cui, H. Zhong, Y. Li, C. Gu, and D. He, “Efficient Blockchain-Based Data Integrity Auditing for Multi-Copy in Decentralized Storage,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 12, pp. 3162–3172, 2023.
- [7] Y. Miao, K. K. Gai, and L. Zhu, “Blockchain-Assisted Multi-Keyword Searchable Provable Data Possession for Cloud Storage,” *Science China Information Sciences*, vol. 69, no. 3, article 132101, 2026.
- [8] Y. Xu, H. Cheng, J. Zheng, X. Zhang, and H. Wang, “MPA: Lightweight and Updatable Integrity Auditing for Decentralized Storage Using Merkle Trees and Polynomial Commitments,” *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 2162–2176, 2026.

Table III: Computational Complexity Comparison (Exponentiations E and Pairings P)

Scheme	Store/Setup	ProofGen	Verify
Yu et al. [5]	mnE	mE	$m(s + c + 4)E$
Zhang et al. [6]	$2mnE$	$(mc + m + 2s - 2)E$	$(mc + m + 3)E + 4P$
Miao et al. [7]	$(m + mn)E$	$(mc + 1)E$	$(m(c + 1) + 2)E + 2P$
Xu et al. [8]	mnE	$m(s - 1)E$	$(mc + 2)E + 2P$
Ours (Proposed)	mnE	$(2ms + m)E$	$(mc + 6m + 1)E + 2P$

Table IV: Communication Complexity Comparison (in bits)

Scheme	Store Phase	ProofGen Phase
Yu et al. [5]	$mns p + mn \mathbb{G}_1 + m h $	$m((s + 1) p + (c + 1) \mathbb{G}_1 + c \log_2 n h)$
Zhang et al. [6]	$mns p + mn \mathbb{G}_1 $	$3 \mathbb{G}_1 + p $
Miao et al. [7]	$mn p + (mn + m) \mathbb{G}_1 $	$ p + 2 \mathbb{G}_1 $
Xu et al. [8]	$mns p + mn \mathbb{G}_1 + mn(\log_2 n + 1) h + m h $	$m(p + (c + 1) \mathbb{G}_1 + c \log_2 n h)$
Ours (Proposed)	$mns p + mn \mathbb{G}_1 + m h $	$m(c + 4) \mathbb{G}_1 + 2m p + mc \log_2 n h $